# Phala Network 개발 도구 및 스크립트 분석

## 1. 개발 도구 개요

Phala Network는 **개발자 친화적인 도구와 스크립트 모음**을 제공하여 로컬 개발 환경 구축, 테스트, 디버깅, 모니터링을 쉽게 할 수 있도록 지원합니다.

## 2. 스크립트 디렉토리 구조

### 2.1 전체 구조

```
scripts/
├── run/ # 로컬 개발 네트워크 실행
├── contrib/ # 기여자용 도구
├── debug-cli/ # 디버깅 CLI 도구
├── js/ # JavaScript 유틸리티
├── prometheus_exporter/ # 메트릭 수집기
├── init.sh # 환경 초기화
├── build 관련 스크립트들
├── 분석 도구들
└── 유틸리티 스크립트들
```

### 2.2 핵심 스크립트 분류

```
# 환경 설정
├── init.sh  # WASM 빌드 환경 초기화
├── coverage-pallets.sh  # 팰릿 코드 커버리지

# 빌드 및 실행
├── run/build.sh  # 전체 프로젝트 빌드
├── run/build-pruntime.sh  # pRuntime 빌드
├── run/node.sh  # 블록체인 노드 실행
├── run/pruntime.sh  # pRuntime 실행
├── run/pherry.sh  # Pherry 릴레이어 실행

# 분석 및 디버깅
├── cloc.sh  # 코드 라인 수 계산
├── dump-metadata.py  # 메타데이터 덤프
├── storage-analyze.py  # 스토리지 분석
├── get-state.sh  # 상태 조회
└── mqdiff.py  # 메시지 큐 차이 분석
```

# 3. 로컬 개발 환경 구축

## 3.1 환경 초기화 (init.sh)

```
#!/usr/bin/env bash
```

```
set -e

echo "*** Initializing WASM build environment"

# CI 환경이 아닌 경우에만 rustup 업데이트
if [ -z $CI_PROJECT_NAME ] ; then
rustup update nightly
rustup update stable
fi

# WASM 타겟 추가
rustup target add wasm32-unknown-unknown --toolchain nightly
```

**기능:**

- Rust 툴체인 업데이트 (nightly, stable)
- WASM 컴파일 타겟 설치
- CI/CD 환경 고려한 조건부 실행

### 3.2 프로젝트 빌드 ([build.sh](build.sh))

```
#!/bin/bash

# Git 서브모듈 초기화 및 업데이트
git submodule update --init --recursive
```

```
# 릴리즈 모드로 전체 프로젝트 빌드
cargo build --release
```

**빌드 결과물:**

- `target/release/phala-node` : 블록체인 노드
- `target/release/pherry` : 릴레이어
- 기타 유틸리티 바이너리들

### 3.3 pRuntime 빌드 ([build-pruntime.sh](build-pruntime.sh))

```
#!/bin/bash

# pRuntime 디렉토리로 이동
cd standalone/pruntime

# SGX 모드 또는 소프트웨어 모드 빌드
if [ "$SGX_MODE" = "HW" ]; then
echo "Building pRuntime in SGX Hardware mode"
make
else
echo "Building pRuntime in Software mode"
SGX_MODE=SW make
fi
```

```
# Gramine 지원 빌드 (선택적)
if [ "$GRAMINE" = "1" ];  then
echo  "Building with Gramine support"
make  gramine
fi
```

# 4. 로컬 네트워크 실행

## 4.1 개발 네트워크 시작 절차

```
# 1. 프로젝트 빌드
./scripts/run/build.sh
./scripts/run/build-pruntime.sh

# 2. pRuntime 상태 초기화
./scripts/run/clear-pruntime.sh

# 3. 3개의 터미널에서 각각 실행
./scripts/run/node.sh  # 터미널 1: 블록체인 노드
./scripts/run/pruntime.sh  # 터미널 2: pRuntime
./scripts/run/pherry.sh  # 터미널 3: 릴레이어
```

### 4.2 블록체인 노드 실행 (node.sh)

```
#!/bin/bash


./target/release/phala-node \
--dev --tmp \ # 개발 모드, 임시 DB
--rpc-methods=Unsafe \ # 모든 RPC 메서드 허용
--rpc-cors=all \ # CORS 모든 오리진 허용
--ws-port  19944 \ # WebSocket 포트
--ws-external \ # 외부 WebSocket 접근 허용
--rpc-port  19933 \ # HTTP RPC 포트
--rpc-external \ # 외부 RPC 접근 허용
```

**특징:**

- 개발 모드로 실행 (Alice가 validator)
- 임시 데이터베이스 사용 (재시작 시 초기화)
- 모든 RPC 메서드 허용 (개발용)
- 외부 접근 허용

### 4.3 pRuntime 실행 (pruntime.sh)

```
#!/bin/bash


cd  standalone/pruntime
```

```
# 환경 변수 설정
export RUST_LOG=${RUST_LOG:-info}
export ROCKET_ADDRESS=0.0.0.0
export ROCKET_PORT=18000

# SGX 모드에 따른 실행
if [ "$SGX_MODE" = "HW" ]; then
echo "Starting pRuntime in SGX Hardware mode"
./bin/pruntime
else
echo "Starting pRuntime in Software mode"
SGX_MODE=SW ./bin/pruntime
fi
```

### 4.4 Pherry 릴레이어 실행 (pherry.sh)

```
#!/bin/bash

# 기본 설정
SUBSTRATE_WS_ENDPOINT=${SUBSTRATE_WS_ENDPOINT:-ws://localhost:19944}
PRUNTIME_ENDPOINT=${PRUNTIME_ENDPOINT:-http://localhost:18000}
MNEMONIC=${MNEMONIC:-//Alice}
```

```
./target/release/pherry \
--substrate-ws-endpoint $SUBSTRATE_WS_ENDPOINT \
--pruntime-endpoint  $PRUNTIME_ENDPOINT \
--mnemonic "$MNEMONIC" \
--fetch-blocks  100 \
--sync-blocks
```

# 5. 분석 및 디버깅 도구

## 5.1 코드 분석 도구

```
# 코드 라인 수 계산 (cloc.sh)
#!/bin/bash
cloc  --exclude-dir=target,node_modules,dist \
--exclude-ext=lock,toml \
--by-file-by-lang \
.

# 결과 예시:
# Rust: 150,000 lines
# TypeScript: 25,000 lines
# JavaScript: 15,000 lines
```

## 5.2 메타데이터 덤프 도구 (dump-metadata.py)

```
#!/usr/bin/env python3
"""
블록체인 런타임 메타데이터를 덤프하는 도구
"""

import json
import sys
from substrateinterface import SubstrateInterface

def  dump_metadata(endpoint="ws://localhost:19944"):
# Substrate 인터페이스 연결
substrate = SubstrateInterface(url=endpoint)
# 메타데이터 조회
metadata = substrate.get_metadata()
# JSON 형태로 출력
print(json.dumps(metadata.value, indent=2, default=str))
# 팰릿 정보 요약
print("\n=== Pallets Summary ===", file=sys.stderr)
for pallet in metadata.value['metadata']['V14']['pallets']:
print(f"- {pallet['name']}: {len(pallet.get('calls', {}).get('type', {}).get('va
file=sys.stderr)
```

```python
if __name__ == "__main__":
endpoint = sys.argv[1] if len(sys.argv) > 1 else "ws://localhost:19944"
dump_metadata(endpoint)
```

### 5.3 스토리지 분석 도구 (storage-analyze.py)

```python
#!/usr/bin/env python3
"""
블록체인 스토리지 사용량 분석 도구
"""

import json
from substrateinterface import SubstrateInterface
from collections import defaultdict

def analyze_storage(endpoint="ws://localhost:19944"):
substrate = SubstrateInterface(url=endpoint)
# 스토리지 키 조회
storage_keys = substrate.query_map("System", "Account")
# 팰릿별 스토리지 사용량 분석
pallet_usage = defaultdict(int)
total_keys = 0
```

```
for key, value in storage_keys:

pallet_name = extract_pallet_name(key)

pallet_usage[pallet_name] += 1

total_keys += 1

# 결과 출력

print(f"Total storage keys: {total_keys}")

print("\nStorage usage by pallet:")

for pallet, count in  sorted(pallet_usage.items(), key=lambda  x: x[1], reverse=

percentage = (count / total_keys) * 100

print(f" {pallet}: {count} keys ({percentage:.1f}%)")



def  extract_pallet_name(storage_key):

# 스토리지 키에서 팰릿 이름 추출

# 실제 구현은 키 형식에 따라 달라짐

return  "Unknown"



if __name__ == "__main__":

analyze_storage()
```

# 6. 메트릭 및 모니터링

## 6.1 Prometheus Exporter

```python
# scripts/prometheus_exporter/exporter.py (추정)
#!/usr/bin/env python3
"""
Phala Network Prometheus 메트릭 수집기
"""

import time
import requests
from prometheus_client import start_http_server, Gauge, Counter
from substrateinterface import SubstrateInterface

# 메트릭 정의
BLOCK_HEIGHT = Gauge('phala_block_height', 'Current block height')
ACTIVE_WORKERS = Gauge('phala_active_workers', 'Number of active workers')
TOTAL_STAKE = Gauge('phala_total_stake', 'Total staked amount')
CONTRACT_CALLS = Counter('phala_contract_calls_total', 'Total contract calls')

class PhalaMetricsCollector:
def __init__(self, substrate_endpoint, pruntime_endpoint):
self.substrate = SubstrateInterface(url=substrate_endpoint)
self.pruntime_endpoint = pruntime_endpoint
def collect_blockchain_metrics(self):
```

```
# 블록 높이
header = self.substrate.get_chain_head()
BLOCK_HEIGHT.set(header['number'])
# 활성 워커 수
workers = self.substrate.query_map("PhalaRegistry", "Workers")
active_count = sum(1  for _, worker in workers if worker['state'] == 'Ready')
ACTIVE_WORKERS.set(active_count)
# 총 스테이킹 금액
total_stake = self.substrate.query("PhalaStakePool", "TotalStake")
TOTAL_STAKE.set(total_stake.value if total_stake else  0)
def  collect_pruntime_metrics(self):
try:
# pRuntime 정보 조회
response = requests.get(f"{self.pruntime_endpoint}/get_info", timeout=5)
info = response.json()
# 컨트랙트 호출 수 (누적)
if  'contract_calls' in info:
CONTRACT_CALLS._value.set(info['contract_calls'])
except Exception as e:
print(f"Failed to collect pRuntime metrics: {e}")
def  run(self, interval=30):
while True:
try:
```

```
self.collect_blockchain_metrics()

self.collect_pruntime_metrics()

time.sleep(interval)

except Exception as e:

print(f"Error collecting metrics: {e}")

time.sleep(interval)



if __name__ == "__main__":

# Prometheus HTTP 서버 시작

start_http_server(8000)

# 메트릭 수집기 시작

collector = PhalaMetricsCollector(

substrate_endpoint="ws://localhost:19944",

pruntime_endpoint="http://localhost:18000"

)

print("Phala metrics exporter started on port 8000")

collector.run()
```

### 6.2 상태 조회 스크립트 (get-state.sh)

```
#!/bin/bash



# 블록체인 상태 조회 스크립트
```

```
ENDPOINT=${1:-ws://localhost:19944}


echo "=== Phala Network Status ==="
echo "Endpoint: $ENDPOINT"
echo


# 체인 정보
echo "Chain Information:"
curl -s -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method":"system_chain"}' \
http://localhost:19933 | jq -r '.result'


# 블록 높이
echo "Current Block:"
curl -s -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method":"chain_getHeader"}' \
http://localhost:19933 | jq -r '.result.number'


# 워커 수
echo "Active Workers:"
# 실제로는 더 복잡한 쿼리 필요
```

```
# pRuntime 상태 (있는 경우)
if curl -s http://localhost:18000/get_info > /dev/null 2>&1; then
echo "pRuntime Status:"
curl -s http://localhost:18000/get_info | jq '.'
fi
```

# 7. 개발 워크플로우 도구

## 7.1 자동화된 개발 환경 설정

```
#!/bin/bash
# scripts/dev-setup.sh (추정)

set -e

echo "🚀 Setting up Phala development environment..."

# 1. 의존성 확인
check_dependencies() {
echo "📋 Checking dependencies..."
# Rust 설치 확인
if ! command -v rustc &> /dev/null; then
echo "❌ Rust not found. Please install Rust first."
```

```
exit 1
fi
# Node.js 확인 (프론트엔드용)
if ! command -v node &> /dev/null; then
echo "⚠️ Node.js not found. Some features may not work."
fi
# Docker 확인 (선택적)
if ! command -v docker &> /dev/null; then
echo "⚠️ Docker not found. Container builds will not work."
fi
echo "✅ Dependencies check completed"
}

# 2. 환경 초기화
setup_environment() {
echo "🔧 Setting up build environment..."
./scripts/init.sh
echo "📦 Installing git hooks..."
# Git hooks 설치 (코드 품질 확인용)
cp scripts/contrib/pre-commit .git/hooks/
chmod +x .git/hooks/pre-commit
}
```

```
# 3. 프로젝트 빌드
build_project() {
echo "🔨 Building project..."
./scripts/run/build.sh
./scripts/run/build-pruntime.sh
echo "✅ Build completed"
}


# 4. 테스트 실행
run_tests() {
echo "🧪 Running tests..."
# 단위 테스트
cargo test --workspace --exclude pruntime
# E2E 테스트 (선택적)
if [ "$SKIP_E2E" != "1" ]; then
cd e2e
yarn install
yarn test
cd ..
fi
echo "✅ Tests completed"
}
```

```
# 메인 실행
main() {
check_dependencies
setup_environment
build_project
if [ "$SKIP_TESTS" != "1" ];  then
run_tests
fi
echo  "🎉 Development environment setup completed!"
echo
echo  "Next steps:"
echo  "1. Start the development network:"
echo  " ./scripts/run/node.sh"
echo  " ./scripts/run/pruntime.sh"
echo  " ./scripts/run/pherry.sh"
echo
echo  "2. Access the network:"
echo  " - Blockchain: ws://localhost:19944"
echo  " - pRuntime: http://localhost:18000"
}


main  "$@"
```

### 7.2 코드 품질 도구

```bash
#!/bin/bash
# scripts/contrib/pre-commit (Git hook)

set -e

echo "🔍 Running pre-commit checks..."

# 1. Rust 코드 포맷팅 확인
echo "📝 Checking Rust formatting..."
if ! cargo fmt --all -- --check; then
echo "❌ Code formatting issues found. Run 'cargo fmt' to fix."
exit 1
fi

# 2. Clippy 린팅
echo "🔧 Running Clippy..."
if ! cargo clippy --all-targets --all-features -- -D warnings; then
echo "❌ Clippy warnings found. Please fix them."
exit 1
fi
```

```
# 3. 테스트 실행 (빠른 테스트만)
echo "🧪 Running quick tests..."
if ! cargo test --lib --bins; then
echo "❌ Tests failed. Please fix them."
exit 1
fi

# 4. 프론트엔드 린팅 (있는 경우)
if [ -d "frontend" ]; then
echo "🎨 Checking frontend code..."
cd frontend/packages/sdk
if ! yarn eslint:ci; then
echo "❌ Frontend linting failed."
exit 1
fi
cd ../../..
fi

echo "✅ All pre-commit checks passed!"
```

이 문서는 Phala Network의 개발 도구와 스크립트들을 다룹니다. 다음 문서에서는 배포 및 운영 가이드를 분석합니다.